GenSpark

Functions & Modular Design

Functions and Modular Design

In Week 1 you wrote programs that ran from top to bottom. That works for small programs, but it breaks down as they grow. The Number Guessing Game already had menus, validation, a timer, and a guess loop in one long file. This article teaches you to break a program into small, named pieces called functions, and to split those pieces across files called modules. It finishes with recursion, a technique where a function calls itself, which you will need for the Hard AI in Tic-Tac-Toe.

Why Functions

A function is a named block of code that you can run whenever you need it. Functions give you four things:

- **Reuse.** Write the code once and use it in many places, instead of copying it.
- **Clarity.** A name like `ask_for_number` tells a reader what a block does without reading it.
- **Testability.** A small function can be checked on its own. You will use this in the pytest article.
- **Easier change.** Fix a bug in one place and every use is fixed.

This follows the DRY principle from the coding best practices article: Don't Repeat Yourself.

Defining and Calling a Function

You define a function with `def` , a name, parentheses, a colon, and an indented body:

python

```
def say_hello():
    print("Hello!")

say_hello()      # call the function
say_hello()      # and again
```

Defining a function does not run it. The code inside only runs when you **call** the function by writing its name followed by parentheses. A function must be defined before the line that calls it runs.

Parameters and Arguments

Most functions need input. The names inside the parentheses of the definition are **parameters**. The values you pass

when you call the function are **arguments**.

python

```
def greet(name):
    print(f"Hello, {name}!")

greet("Alex")      # Hello, Alex!
greet("Sam")       # Hello, Sam!
```

A function can take several parameters, separated by commas:

python

```
def add_numbers(a, b):
    return a + b
```

Arguments are matched to parameters by position, so `add_numbers(3, 4)` sets `a` to 3 and `b` to 4. If the counts do not match, Python tells you:

```
TypeError: add_numbers() missing 1 required positional argument: 'b'
TypeError: add_numbers() takes 2 positional arguments but 3 were given
```

Keyword arguments

You can pass arguments by name, in any order. This makes calls easier to read when there are several values:

python

```
def greet(name, greeting):
    return f"{greeting}, {name}!"

greet(greeting="Hi", name="Sam")     # Hi, Sam!
```

Default values

A parameter can have a default, which is used when the caller leaves it out:

python

```
def greet(name, greeting="Hello"):
    return f"{greeting}, {name}!"

greet("Alex")                # Hello, Alex!
greet("Alex", "Welcome")     # Welcome, Alex!
```

Parameters with defaults must come after the ones without. The Tic-Tac-Toe lab uses this to let a caller pass in a seeded random generator for testing, and use a fresh one otherwise.

Return Values

A function can send a value back to the code that called it with `return`.

python

```
def add_numbers(a, b):
    return a + b

total = add_numbers(3, 4)
print(total)      # 7
```

return is not print

This is the most common confusion about functions. `print()` shows something on the screen. `return` hands a value back to the program so it can use it.

python

```
def show_sum(a, b):
    print(a + b)

result = show_sum(3, 4)      # prints 7
print(result)                # None
```

`show_sum` printed 7, but it returned nothing, so `result` holds `None`. If you want to use the answer in later code, return it. Print it only at the point where you want the user to see it.

A function with no `return` statement returns `None` automatically.

return ends the function

The moment Python reaches `return`, the function stops. Lines after it never run:

python

```
def judge_guess(guess, secret):
    if guess < secret:
        return "too low"
    if guess > secret:
        return "too high"
    return "correct"
```

This works without `elif`, because each `return` leaves the function. If the first condition is true, the function never reaches the others.

Returning more than one value

A function can return several values separated by commas. Python packs them together, and you can unpack them into separate variables:

python

```
def min_and_max(a, b):
    return min(a, b), max(a, b)

low, high = min_and_max(9, 4)
print(low, high)      # 4 9
```

This is useful for something like a difficulty menu that returns the low number, the high number, and the attempts allowed.

Scope: Where Variables Live

A variable created inside a function is **local**. It exists only while the function runs, and the rest of your program cannot see it.

python

```
def area(width, height):
    result = width * height
    return result

area(2, 3)
print(result)      # NameError: name 'result' is not defined
```

This protects your code. Two functions can both use a variable called `total` without interfering with each other. Parameters are local too.

A function can **read** a variable defined outside it, but if it tries to **change** one, Python treats the name as a new local variable:

python

```
count = 0

def bump():
    count = count + 1      # tries to read a local count that does not exist yet

bump()
```

```
UnboundLocalError: cannot access local variable 'count' where it is not associated with a v
```

The clean fix is to pass the value in and return the new one:

python

```
def bump(count):
    return count + 1

count = bump(count)
```

Python does have a `global` keyword that lets a function change an outer variable, but avoid it. Functions that depend on hidden outside variables are hard to understand and hard to test. Pass values in, and return values out.

Settings that never change, like `TIME_LIMIT = 60`, are fine to read from inside a function. They are constants, and the `ALL_CAPS` name tells the reader so.

Functions That Call Functions

Functions can call other functions. This is how you build a program from small parts. Here is the validation code from the loops article, packaged as a function:

python

```
def ask_for_number(prompt, low, high):
    """Keep asking until the user types a whole number from low to high. Return it."""
    while True:
        text = input(prompt).strip()
        if not text.isdigit():
            print("Please type a whole number.")
            continue
        number = int(text)
        if number < low or number > high:
            print(f"Please pick a number from {low} to {high}.")
            continue
        return number
```

The `return` inside the loop leaves both the loop and the function. Any part of your program can now say `guess = ask_for_number("Guess: ", 1, 100)` and get a clean, valid whole number. The checking code lives in one place.

Docstrings and Type Hints

A **docstring** is a string in triple quotes placed directly under the `def` line. It describes what the function does. Tools like PyCharm show it when you hover over a call.

python

```
def circle_area(radius):
    """Return the area of a circle with the given radius."""
    return 3.14159 * radius ** 2
```

Write one sentence that says what the function returns or does. Start with a verb. You do not need to explain how, since the code shows that.

Type hints are optional labels that say what types a function expects and returns:

python

```
def add_numbers(a: int, b: int) -> int:
    return a + b
```

Python does not enforce them. `add_numbers("a", "b")` still runs and returns `"ab"`. Their value is for readers and for tools like PyCharm, which warn you when you pass the wrong type. You will see them in professional code, and you are welcome to use them.

Designing Good Functions

Give each function one job. If you need the word "and" to describe what a function does ("it asks for a guess and checks it and prints the result"), split it.

Name functions with verbs. `ask_for_number`, `judge_guess`, `play_round`, `calculate_total`. A good name saves a comment.

Keep them short. If a function does not fit on one screen, it probably has more than one job.

Do not repeat yourself. If you copy the same lines into two places, make them a function.

Prefer return values over side effects. A function that only takes inputs and returns an output is easy to test. A function that prints, asks for input, or changes outside variables is harder.

Separate the logic from the input and output

This is the most useful design habit in this article, and the Tic-Tac-Toe lab requires it. Compare:

python

```
def check_guess():                      # does everything
    guess = int(input("Guess: "))
    if guess < 42:
```

```
        print("Too low")
    elif guess > 42:
        print("Too high")
    else:
        print("Correct")
```

with:

python

```
def judge_guess(guess, secret):          # only decides
    """Return "too low", "too high", or "correct"."""
    if guess < secret:
        return "too low"
    if guess > secret:
        return "too high"
    return "correct"
```

The first version cannot be tested without a person typing at the keyboard, and it has the secret number hard-coded. The second version takes its inputs as parameters and returns the answer, so you can check it instantly:

python

```
print(judge_guess(10, 42))     # too low
print(judge_guess(90, 42))     # too high
print(judge_guess(42, 42))     # correct
```

The rule of thumb: keep the **decisions** in functions that take values and return values, and keep `input()` and `print()` in a thin outer layer. In the pytest article, you will test the decision functions automatically.

Putting a Program Together

Here is the guessing game round from Week 1, rebuilt with functions. Notice how short each function is:

python

```
import random


def ask_for_number(prompt, low, high):
    """Keep asking until the user types a whole number from low to high. Return it."""
    while True:
        text = input(prompt).strip()
        if not text.isdigit():
            print("Please type a whole number.")
            continue
        number = int(text)
        if number < low or number > high:
```

```
        if number < low or number > high:
            print(f"Please pick a number from {low} to {high}.")
            continue
        return number


def judge_guess(guess, secret):
    """Return "too low", "too high", or "correct"."""
    if guess < secret:
        return "too low"
    if guess > secret:
        return "too high"
    return "correct"


def play_round(low, high, max_attempts, secret=None):
    """Play one round. Return True if the player wins, False if not."""
    if secret is None:
        secret = random.randint(low, high)

    attempts_used = 0
    while attempts_used < max_attempts:
        guess = ask_for_number(f"Guess a number from {low} to {high}: ", low, high)
        attempts_used += 1

        result = judge_guess(guess, secret)
        if result == "correct":
            print(f"Correct! You needed {attempts_used} attempts.")
            return True

        print(result.capitalize())
        print(f"Attempts remaining: {max_attempts - attempts_used}")

    print(f"Out of attempts. The number was {secret}.")
    return False


def main():
    play_round(1, 100, 8)


main()
```

Three things to notice:

- `play_round` accepts an optional `secret`. In normal play it picks a random number. In a test you can pass a known one. This replaces the `DEBUG_SECRET` setting from Week 1, and it is cleaner because no setting needs resetting.
- `play_round` returns `True` or `False`, so whatever called it can decide what to do next, such as offering

another round.

- `main()` is a short summary of the whole program. A reader who wants the big picture reads that first.

Splitting Code Across Files: Modules

A Python file is a **module**. As programs grow, you move related functions into their own files and `import` them where needed, just as you import `random` and `time`.

Suppose `helpers.py` contains:

python

```
def add_numbers(a, b):
    """Return the sum of a and b."""
    return a + b
```

and `main.py` is in the same folder:

python

```
import helpers
from helpers import add_numbers

print(helpers.add_numbers(3, 4))     # 7
print(add_numbers(10, 20))           # 30
```

You can import a whole module and use its name as a prefix (`helpers.add_numbers`), or import specific functions by name. Run the program from the folder that holds both files, with `python3 main.py`. PyCharm does this for you when you run from the project.

The name guard

When Python imports a module, it runs every line of that file. If `helpers.py` had a stray `print()` at the top level, importing it would print that text. To include code that should only run when you start the file directly, use this pattern:

python

```
def add_numbers(a, b):
    return a + b

if __name__ == "__main__":
    print(add_numbers(2, 3))
```

Python sets the special variable `__name__` to `"__main__"` when you run a file directly, and to the module's name when another file imports it. So the block under the `if` runs for `python3 helpers.py` but not for `import`

`helpers`. Put your `main()` call under this guard:

python

```
if __name__ == "__main__":
    main()
```

This lets you import your functions into a test file without accidentally starting the whole game. You will rely on this in the pytest article.

Module rules

- **Don't name your file after a standard library module.** A file called `random.py` or `time.py` hides the real one.
- **Avoid circular imports.** If `a.py` imports `b.py` and `b.py` imports `a.py`, Python fails or behaves strangely. Organize files so imports go one way: low-level files first, higher-level files importing them.
- **Group by purpose.** Put the code that does one kind of thing in one file.

A preview of the Tic-Tac-Toe layout

The Tic-Tac-Toe lab uses exactly this idea. Each file has one job, and imports flow one way:

File	Job
`board.py`	The board: placing marks, detecting a win
`ai.py`	The computer's move choices
`game.py`	Turns, timing, and forfeits
`main.py`	Menus and printing, the only file that talks to the user

`main.py` imports `game.py`, which imports `board.py` and `ai.py`. Nothing imports `main.py`. That one-way flow is what keeps the project easy to understand.

Recursion

A **recursive** function is a function that calls itself. This sounds strange, but it is a natural way to solve problems that contain smaller versions of themselves. You need it for the Hard AI in Tic-Tac-Toe.

Every recursive function has two parts:

1. **A base case**: a situation simple enough to answer directly, with no further call. This stops the recursion.
2. **A recursive case**: the function calls itself with a smaller or simpler version of the problem.

Here is a countdown:

python

```python
def countdown(n):
    if n == 0:                    # base case
        print("Liftoff")
        return
    print(n)
    countdown(n - 1)              # recursive case: a smaller problem

countdown(3)
```

Output:

```
3
2
1
Liftoff
```

Following the calls

Factorial is the classic example. `5!` means `5 x 4 x 3 x 2 x 1`:

python

```python
def factorial(n):
    if n <= 1:                    # base case
        return 1
    return n * factorial(n - 1)   # recursive case

print(factorial(5))               # 120
```

Here is what happens. Each call waits for the call below it to finish:

```
factorial(5)
 = 5 * factorial(4)
 = 5 * (4 * factorial(3))
 = 5 * (4 * (3 * factorial(2)))
 = 5 * (4 * (3 * (2 * factorial(1))))
 = 5 * (4 * (3 * (2 * 1)))        # base case reached: factorial(1) is 1
 = 120
```

Python keeps a list of the waiting calls, called the **call stack**. Once the base case returns 1, each waiting call finishes in turn, from the inside out.

Another small example adds up the digits of a number:

python

```
def sum_digits(n):
    if n < 10:
        return n
    return n % 10 + sum_digits(n // 10)

print(sum_digits(2468))     # 20
```

`n % 10` is the last digit, and `n // 10` is the number without it. Each call handles one digit and passes the rest along.

Without a base case

A recursive function with no base case, or one that never reaches it, calls itself forever. Python stops it after about 1000 nested calls:

python

```
def forever(n):
    return forever(n + 1)

forever(0)
```

```
RecursionError: maximum recursion depth exceeded
```

If you see this, check two things: does the function have a base case, and does each call move closer to it?

Recursion that makes choices

Recursion is most powerful when each call has to try several options. This is how a computer plays a game. Here is a simple game called take-away. A pile of stones sits between two players. On your turn you take 1 or 2 stones, and **whoever takes the last stone wins**.

Can the player to move force a win? The function tries each move and asks the same question about the opponent:

python

```
def can_win(stones):
    """Return True if the player to move can force a win."""
    if stones == 0:
        return False                    # the opponent took the last stone, so we lost
    for take in (1, 2):
        if take <= stones and not can_win(stones - take):
            return True                 # this move leaves the opponent in a losing posi
    return False                        # every move leaves the opponent winning
```

```
for stones in range(0, 10):
    print(stones, can_win(stones))
```

The output shows that the player to move loses with 0, 3, 6, and 9 stones, and wins with every other number. Read the function as a plan:

- If I can make a move that leaves my opponent in a losing position, I win.
- If every move leaves my opponent in a winning position, I lose.
- To judge the opponent's position, I ask the same question again, from their point of view.

This "assume the opponent plays their best, then pick my best response" pattern is the idea behind **minimax**, the algorithm for your Hard Tic-Tac-Toe AI. The Tic-Tac-Toe version is bigger, because it scores positions and works on a board instead of a number, but the shape is the same: a base case where the game is over, and a loop over possible moves that calls the function again for the opponent.

Recursion or a loop?

Anything you can write with recursion you can also write with a loop, and often a loop is simpler and faster. Use recursion when the problem naturally breaks into smaller copies of itself, as with game trees, nested folders, and tree-shaped data. Use a loop for simple repetition. Very deep recursion is slow and runs into the limit, so recursion is not a replacement for loops.

Common Mistakes

Mistake	What happens	Fix
Defining a function but never calling it	Nothing happens	Call it with parentheses: `play_round()`
Leaving off the parentheses on a call	The function is not run	Write `main()`, not `main`
Printing inside a function when you meant to return	The result is `None` when used later	Use `return` for values
Using `return` and expecting the code below to run	Lines after `return` are skipped	Put anything that must run before `return`
Calling with the wrong number of arguments	`TypeError` about missing or extra arguments	Match the parameters
Changing an outside variable inside a function	`UnboundLocalError`	Pass the value in and return the new one
Using a local variable outside its function	`NameError`	Return the value and store it
A function that does several jobs	Hard to read and test	Split it, one job each
Putting `input()` and `print()` in	Cannot test without typing	Keep decisions in functions that return

every function		values
Stray top-level code in a module	It runs when another file imports it	Put it under `if __name__ == "__main__":`
Two files importing each other	Import errors	Make imports flow one way
A recursive function with no base case	`RecursionError`	Add a base case that is always reached
A recursive call that does not get smaller	`RecursionError`	Make each call closer to the base case

Try It Yourself

Create a file called `functions_practice.py` and run it with `python3 functions_practice.py`. Try these:

1. Write `greet(name)` that returns a greeting. Call it and print the result.
2. Write `add_numbers(a, b)` that returns the sum. Then call it with only one argument and read the `TypeError`.
3. Write a function `show_sum(a, b)` that prints the sum but does not return it. Call it, store the result in a variable, and print the variable. Explain what you see.
4. Write `greet(name, greeting="Hello")`. Call it with and without the second argument, and once with keyword arguments in reverse order.
5. Write `min_and_max(a, b)` that returns two values, and unpack them into two variables.
6. Write a function that tries to change a global variable. Read the `UnboundLocalError`, then fix it by passing the value in and returning the new one.
7. Write `is_even(number)` that returns `True` or `False`. Use it in a loop that prints each number from 1 to 10 and whether it is even.
8. Write `calculate_grade(score)` that returns a letter grade, using the thresholds from the decisions article. Keep `input()` and `print()` out of it. Then write a second function `main()` that asks for a score and prints the grade.
9. Take your Number Guessing Game and move the input checking into one function. Make sure the game still behaves the same. (This is the optional functions goal in that lab.)
10. Create `helpers.py` with two functions and a `main.py` that imports them. Add a stray `print()` at the top of `helpers.py`, notice that it runs on import, and then fix it with the `__name__` guard.
11. Write a recursive `countdown(n)` and a recursive `factorial(n)`. Then call `factorial(5000)` and read the `RecursionError`.
12. Write a recursive `sum_to(n)` that returns `1 + 2 + ... + n`. Check it against `n * (n + 1) // 2`.
13. Change `can_win` so players may take 1, 2, or 3 stones. Predict which pile sizes lose before you run it, then check.

Why This Matters for an FDE

Real systems are too large to hold in your head, so engineers cut them into pieces that each do one clear job. Functions and modules are the first tools for doing that. When you join a client project, you will spend far more time reading other people's code than writing your own, and well-named, well-separated functions are what make that possible. Separating decisions from input and output also determines whether code can be tested automatically, which is what the rest of this week is about.

Comprehension Check

1. What is the difference between a parameter and an argument?
2. What does a function return when it has no `return` statement? Why does `result = show_sum(3, 4)` leave `result` as `None` when `show_sum` prints the answer?
3. Why does `count = count + 1` inside a function raise an `UnboundLocalError`, and what is the cleaner fix?
4. Explain why `judge_guess(guess, secret)` is easier to test than a function that asks for input, compares the guess to 42, and prints the hint.
5. What does `if __name__ == "__main__":` do, and what problem does it prevent?
6. Every recursive function needs two parts. What are they, and what happens if the first is missing?
7. In `can_win`, why does the function call itself with `stones - take`, and what does `not can_win(stones - take)` mean in plain words?
8. Give one case where a loop is better than recursion, and one where recursion is a natural fit.

← Previous: Team 5: Meeting Scheduler: Members: Harshitha Reddy Gundavarapu, Tejdeep Pathipati, Vedanshu Daxesh Pate

Next: Lists and Nested Lists (needed for the Tic-Tac-Toe board) →

0 comments so far

6

Add a comment...

Comment cannot be empty

Comment

No comments yet. Be the first to start the discussion!